

Design and Analysis of Algorithms

Lecture 9: Introduction to Graphs





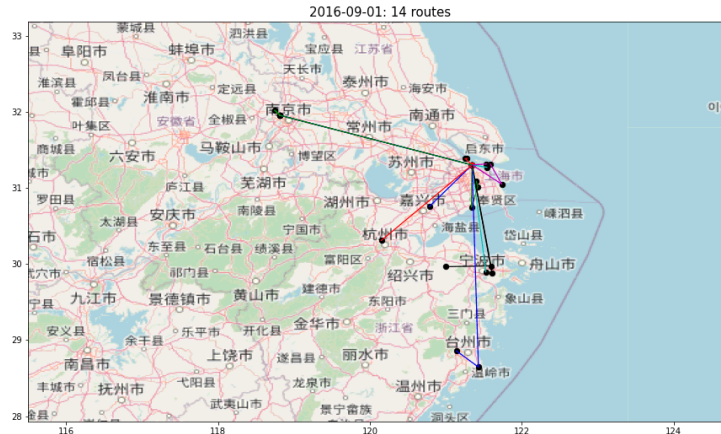
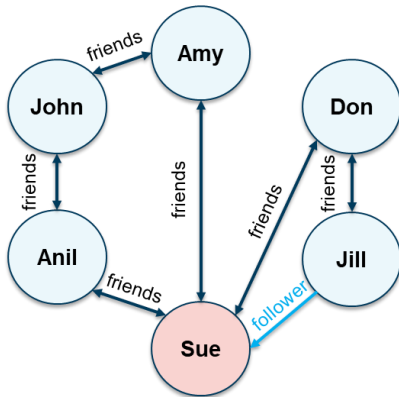
Table of Contents

- What is a graph?
- Problems related to Graph
- Graph Traversal
- Some graph traversal algorithms:

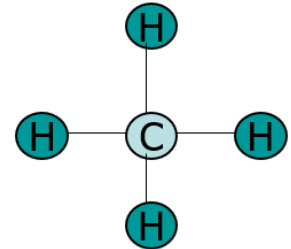


Some Applications of Graphs

- Social networks (Facebook friends)
- Transportation systems (roads, flights)
- Molecular structures

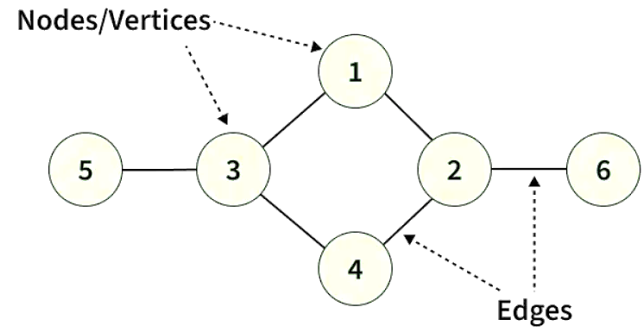


Molecular Structure



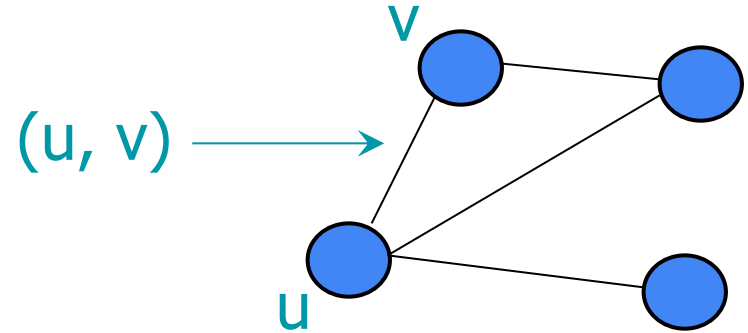
What is a graph?

- Graphs represent the relationships among data items.
- A graph G consists of:
 - a set V of nodes (vertices).
 - a set E of edges: each edge connects two nodes.
- Each node represents an item.
- Each edge represents the relationship between two items.



Formal Definition of graph

- The set of nodes is denoted as V .
- For any nodes u and v , if u and v are connected by an edge, such edge is denoted as (u, v) .
- The set of edges is denoted as E .
- A graph G is defined as a pair (V, E) .



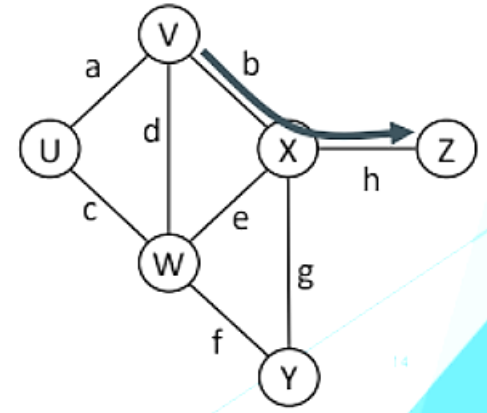
Graph Terminology

- Degree of a vertex
- Path
- Reachability and connectedness
- Loops and cycles



Graph Terminology: Path

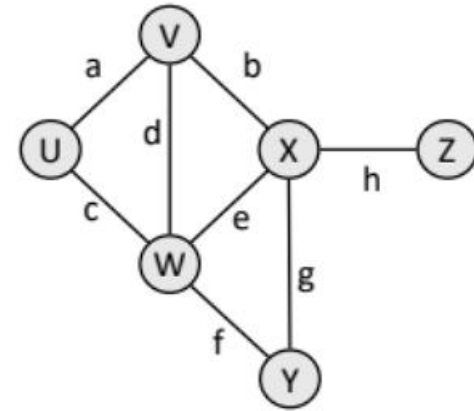
- **Path:** A path from vertex V to Z is a sequence of edges that can be followed starting from a to reach b .
 - ✓ can be represented as vertices visited, or edges taken
 - ✓ example, one path from V to Z : $\{b, h\}$ or $\{V, X, Z\}$
- **Path length:** Number of vertices or edges contained in the path.
- **Neighbor or adjacent:** Two vertices connected directly by an edge.
 - ✓ example: V and X



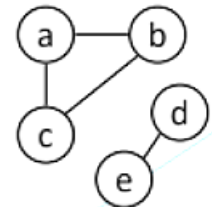
Graph Terminology: Reachability and connectedness

- **Reachable:** Vertex U is reachable from V if a path exists from U to V .
- **Connected Graph:** A graph is connected if every vertex is reachable from any other.
- **Complete Graph:** When every vertex has an edge to every other vertex.

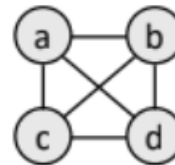
Connected Graph



Is this Connected Graph?

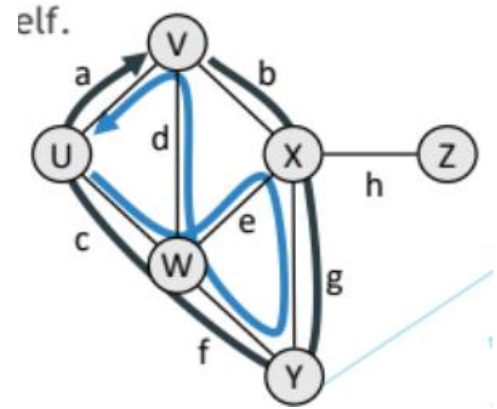


Strongly Connected Graph



Graph Terminology: Loops and cycles

- **Cycle:** A path that begins and ends at the same node.
 - ✓ example: {b, g, f, c, a} or {V, X, Y, W, U, V}.
 - ✓ example: {c, d, a} or {U, W, V, U}.
- **Acyclic graph:** One that does not contain any cycles.
- **Loop:** An edge directly from a node to itself.
 - ✓ Many graphs don't allow loops.



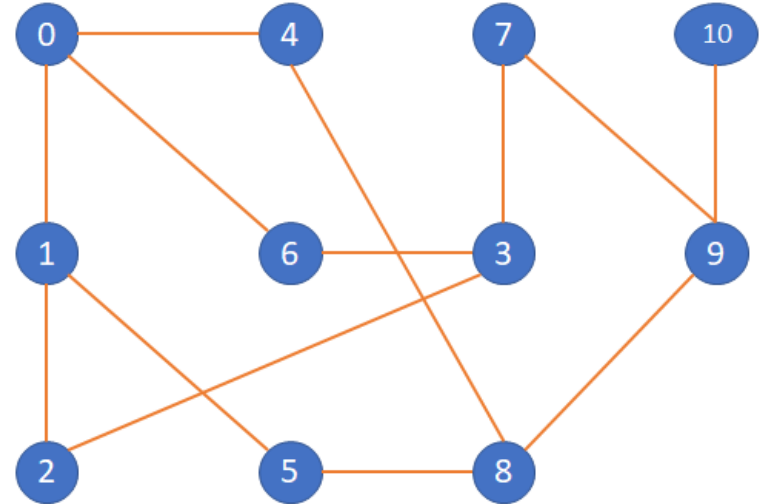
Graph Classification

- **By Direction:**
 - Undirected graphs
 - Directed graphs (Digraphs)
- **By Edge Weights:**
 - Unweighted graphs
 - Weighted graphs
- **Special Types:**
 - Binary Tree
 - DAG (Directed Acyclic Graph)



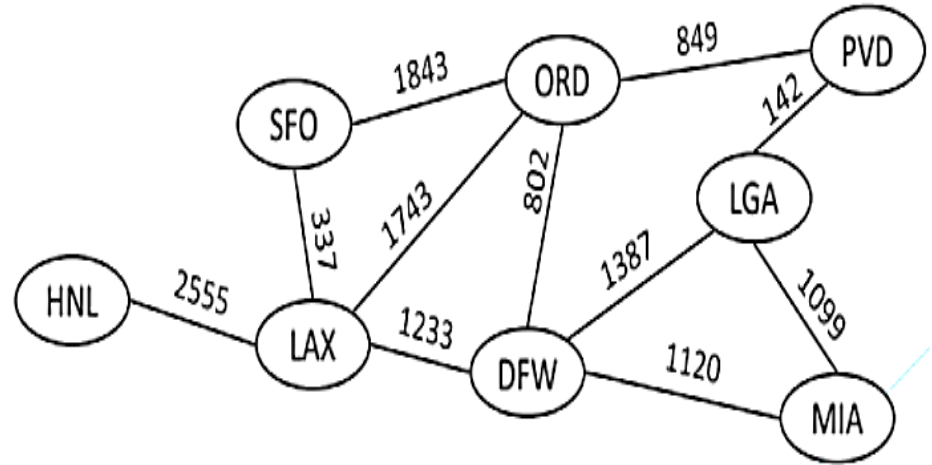
Graph Classification: Unweighted graphs

- **Edges have NO weights** (all edges are equal).
- **All edges = 1 unit** distance/cost.
- **Example:** Facebook friendships.



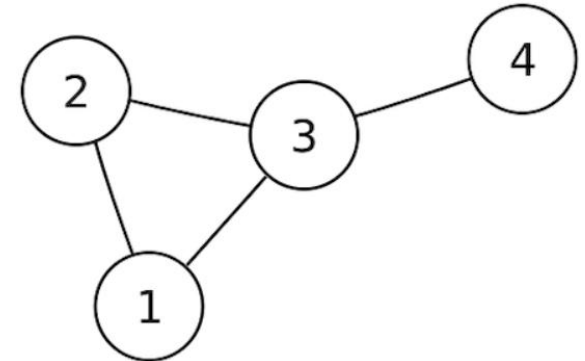
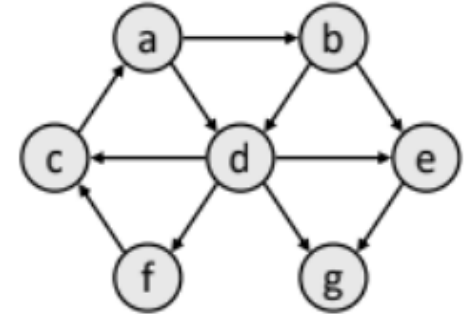
Graph Classification: Weighted graphs

- **Weight:** Cost associated with a given edge. Most graphs do not allow negative weights.
- **Example:** graph of airline flights, weighted by miles between cities:



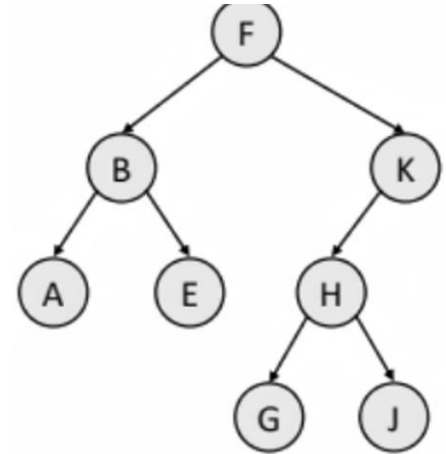
Graph Classification: Directed and Undirected graphs

- **Directed graph ("digraph"):** One where edges are one-way connections between vertices. A digraph can be weighted or unweighted. Used for: Twitter follows (A follows B" doesn't mean "B follows A").
- **Undirected Graph:** Edges have NO direction (bidirectional). If A connected to B, then B connected to A. Used for: Facebook friendships("A is friends with B" implies "B is friends with A").



Graph Classification: Binary Tree

- **A binary tree is a graph with some restrictions:**
 - ✓ The tree is an unweighted, directed, acyclic graph (DAG).
 - ✓ Each node's in-degree is at most 1, and out-degree is at most 2.
 - ✓ There is exactly one path from the root to every node.



Graph Representation in Computers

- **Adjacency matrix**

- Represent a graph using a two-dimensional array.

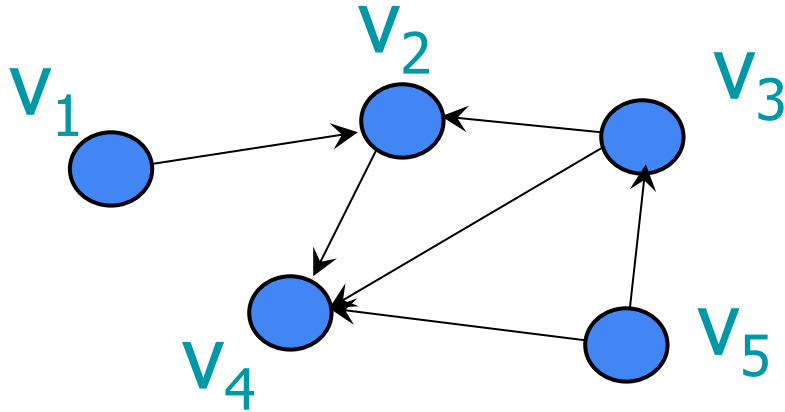
- **Adjacency list**

- Represent a graph using n linked lists where n is the number of vertices.



Adjacency matrix for directed graph

$\text{Matrix}[i][j] = 1$ if $(v_i, v_j) \in E$
 0 if $(v_i, v_j) \notin E$

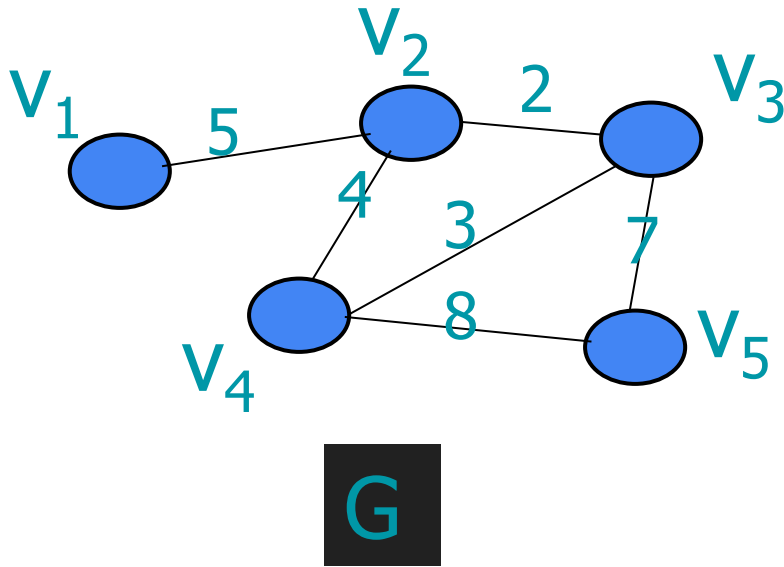


G

		1	2	3	4	5
		v_1	v_2	v_3	v_4	v_5
1	v_1	0	1	0	0	0
2	v_2	0	0	0	1	0
3	v_3	0	1	0	1	0
4	v_4	0	0	0	0	0
5	v_5	0	0	1	1	0

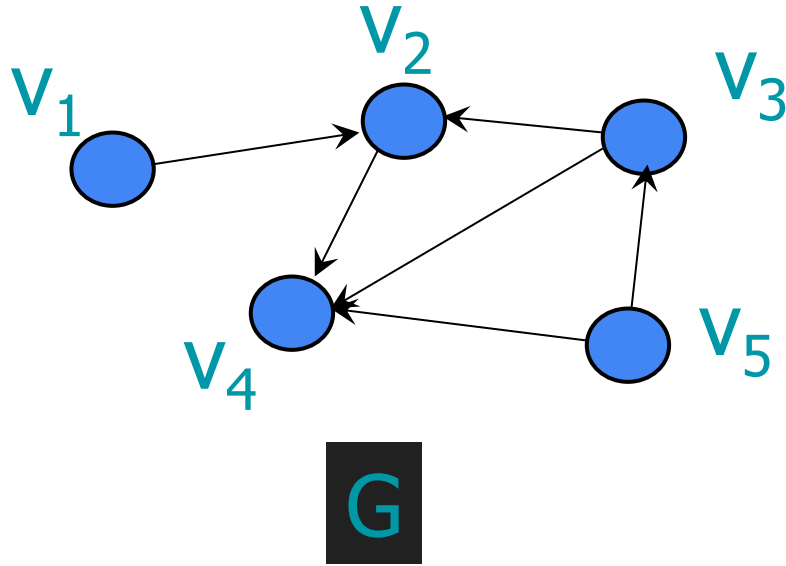
Adjacency matrix for weighted undirected graph

$\text{Matrix}[i][j] = \begin{cases} w(v_i, v_j) & \text{if } (v_i, v_j) \in E \text{ or } (v_j, v_i) \in E \\ \infty & \text{otherwise} \end{cases}$



		1	2	3	4	5
		v_1	v_2	v_3	v_4	v_5
1	v_1	∞	5	∞	∞	∞
2	v_2	5	∞	2	4	∞
3	v_3	∞	2	∞	3	7
4	v_4	∞	4	3	∞	8
5	v_5	∞	∞	7	8	∞

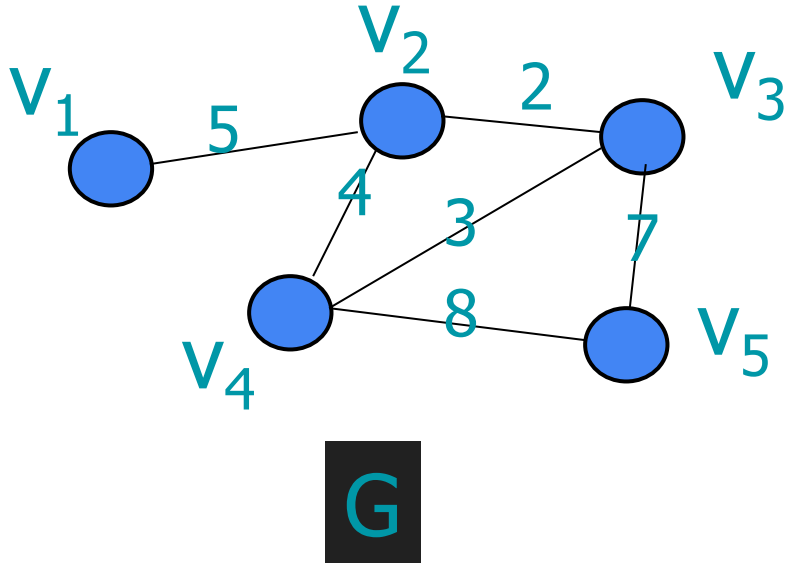
Adjacency list for directed graph



1	v_1	$\rightarrow$	v_2
2	v_2	$\rightarrow$	v_4
3	v_3	$\rightarrow$	$v_2 \rightarrow v_4$
4	v_4		
5	v_5	$\rightarrow$	$v_3 \rightarrow v_4$



Adjacency list for weighted undirected graph

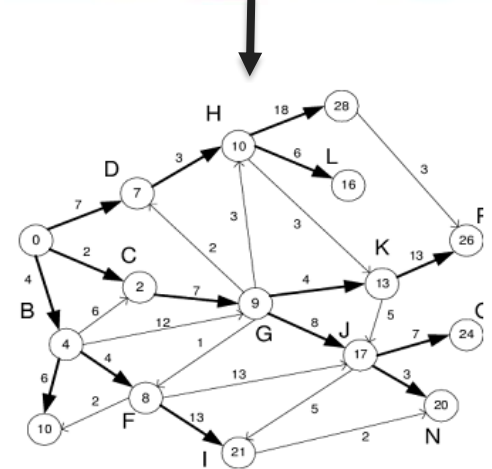


1	v_1	$\rightarrow v_2(5)$			
2	v_2	$\rightarrow v_1(5)$	$\rightarrow v_3(2)$	$\rightarrow v_4(4)$	
3	v_3	$\rightarrow v_2(2)$	$\rightarrow v_4(3)$	$\rightarrow v_5(7)$	
4	v_4	$\rightarrow v_2(4)$	$\rightarrow v_3(3)$	$\rightarrow v_5(8)$	
5	v_5	$\rightarrow v_3(7)$	$\rightarrow v_4(8)$		



Why Graphs Matter in Algorithms?

- **Modeling power:** Represent complex relationships.
- **Algorithmic efficiency:** Specialized graph algorithms are often optimal.
- **Universality:** Many problems reduce to graph problems.



Problems related to Graph

- **Graph Traversal**
- Spanning Tree
- Minimum Spanning Tree
- Shortest Path



The Traversal Challenge

- **Traversal is Nontrivial**
 - ✓ How to systematically visit all vertices in an unpredictable structure?
- **No natural order** (unlike arrays)
- **Cycles can trap us** in infinite loops
- **Disconnected components** may exist



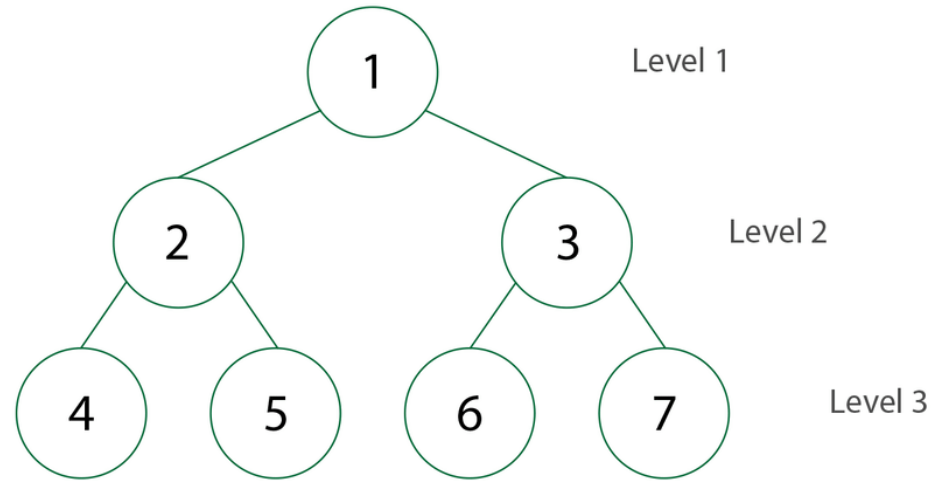
Traversal Technique of Graph

- Traversal techniques are used to visit all the vertices of a graph systematically. These methods help to explore the graph completely and are useful for solving many graph-related problems.
- **Two basic graph traversal algorithms:**
 - ✓ Breadth-first-search (BFS)
 - ✓ Depth-first-search (DFS)

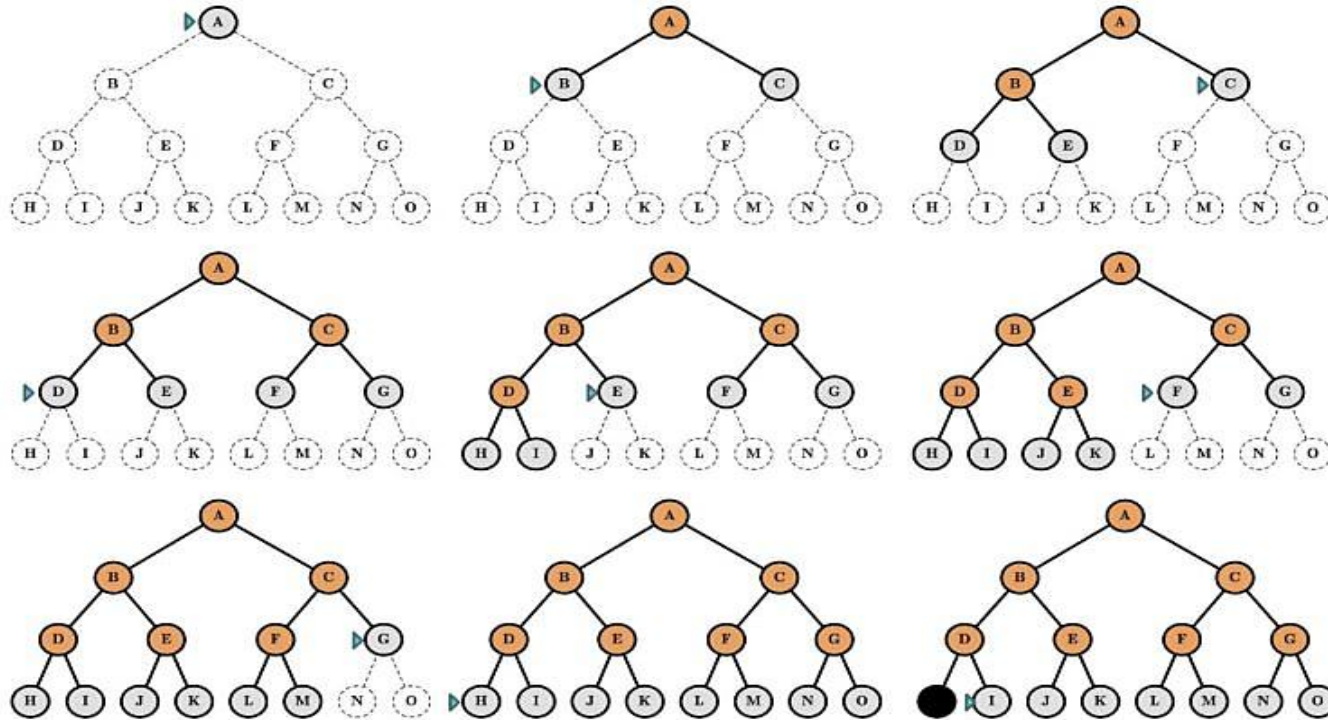


Breadth-first-search (BFS)

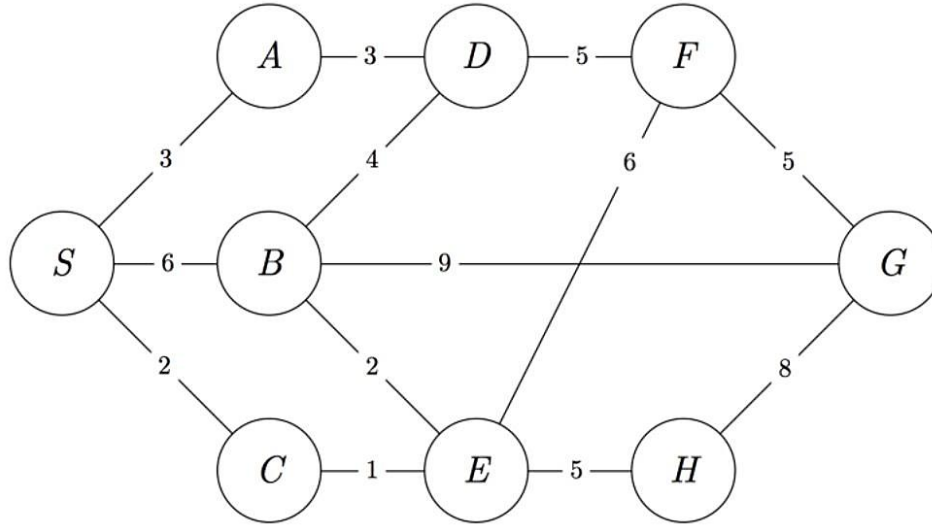
- After visit node v , BFS strategy visits every node adjacent to v before visiting any other nodes.
- Avoiding Repeated Nodes by Keep tracking the generated nodes. If the new node already exists, then discard the node.
- Uses a queue (First input first output (FIFO)).



Breadth-first-search (BFS)



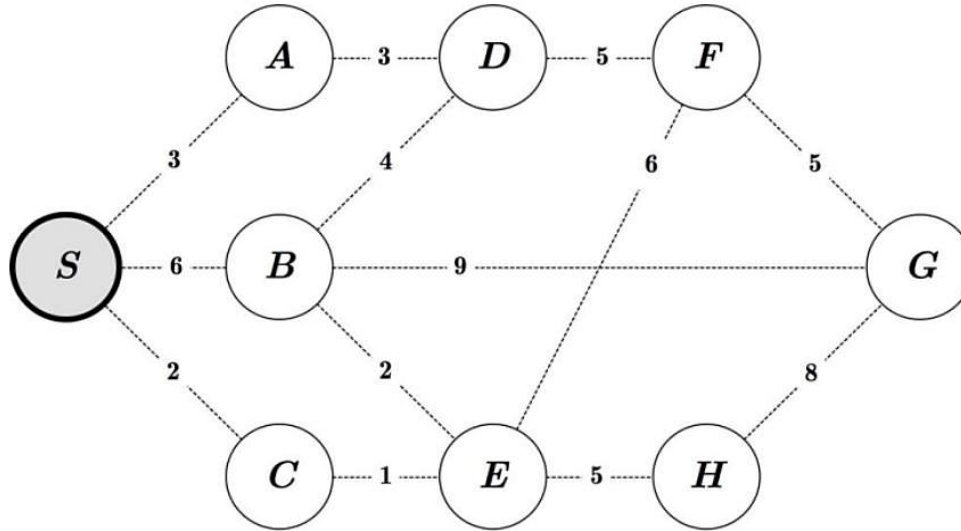
Breadth-first-search (BFS)



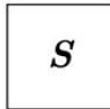
Question: What is the **order of visits of the nodes** and the **path returned** by BFS? Start node S and goal node G.



Breadth-first-search (BFS)



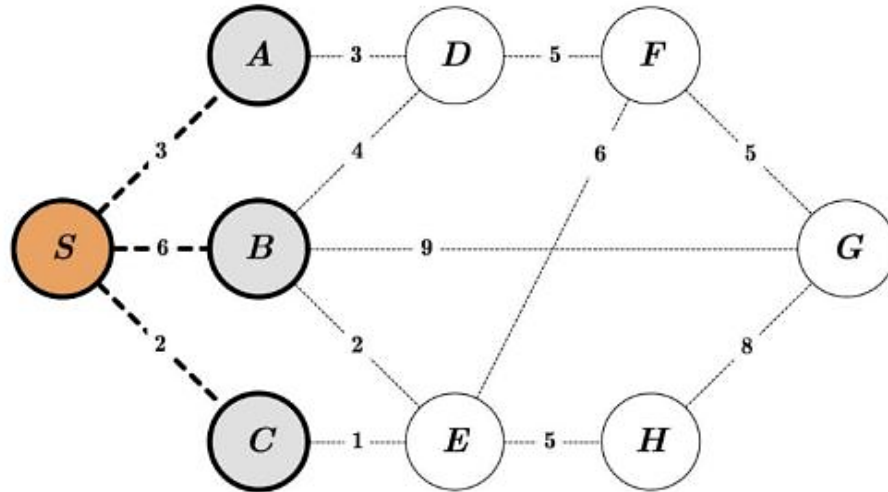
Queue:



Order of Visit:



Breadth-first-search (BFS)



Queue:

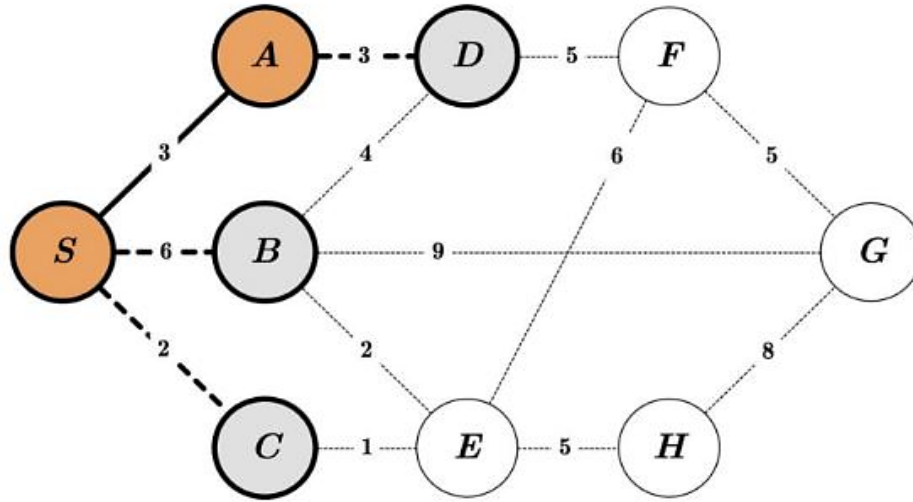
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>
----------	----------	----------	----------

Order of Visit:

S



Breadth-first-search (BFS)



Queue:

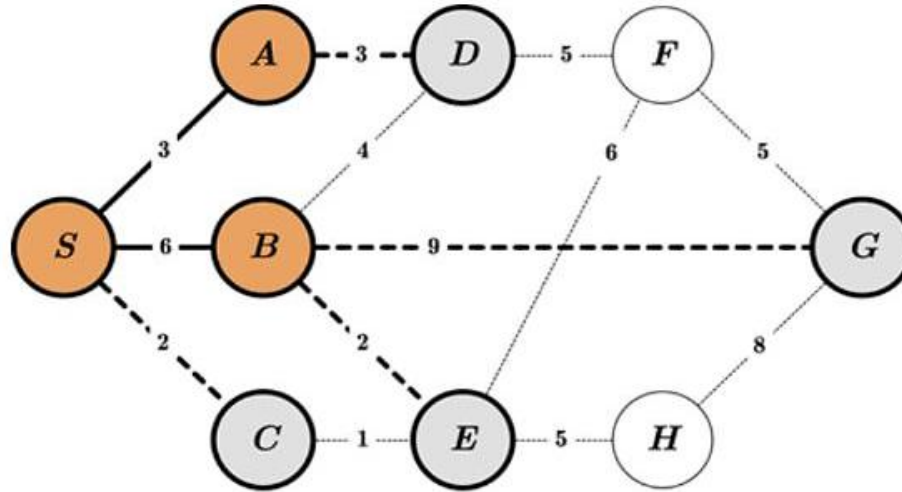
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
----------	----------	----------	----------	----------

Order of Visit:

S *A*



Breadth-first-search (BFS)



Queue:

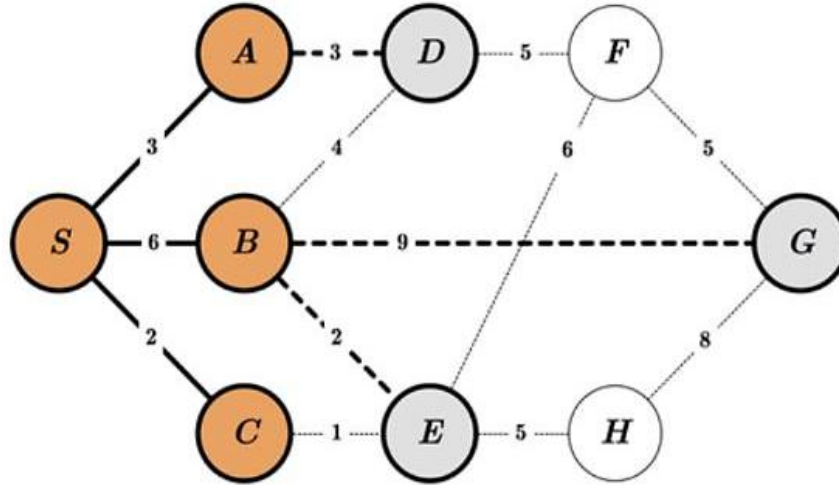
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>G</i>	<i>E</i>
----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *B*



Breadth-first-search (BFS)



Queue:

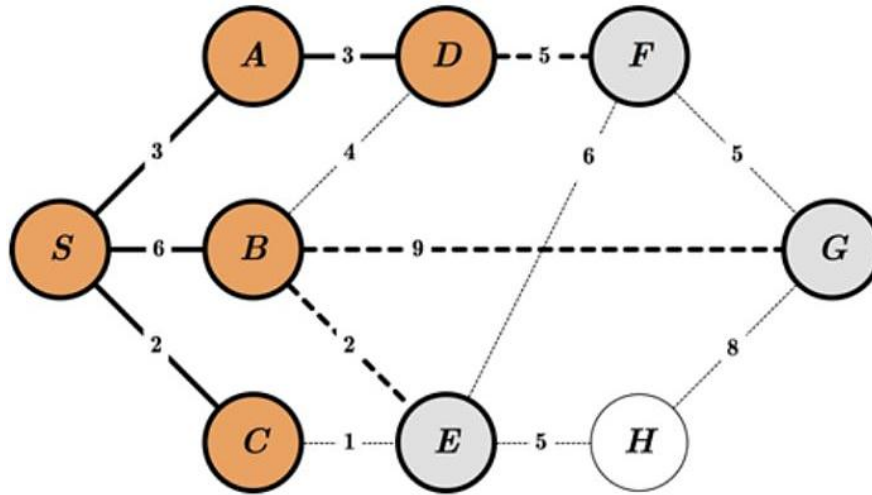
S	A	B	C	D	G	E
---	---	---	---	---	---	---

Order of Visit:

S A B C



Breadth-first-search (BFS)



Queue:

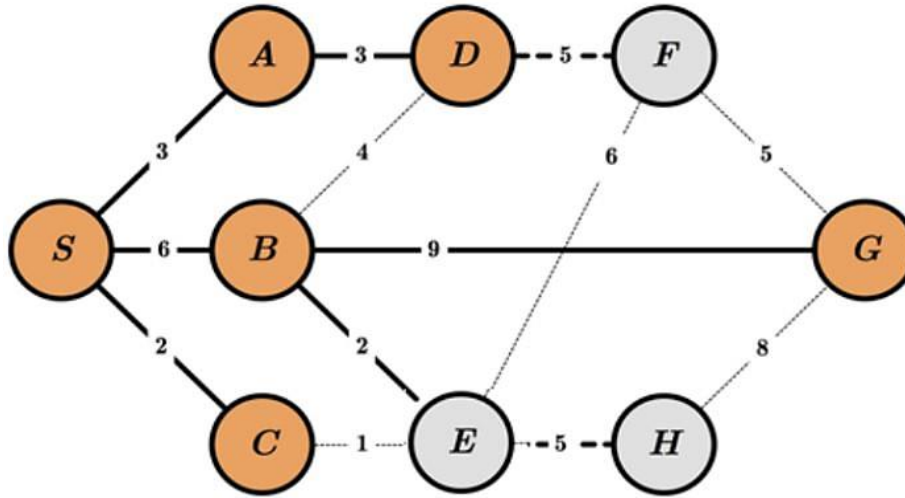
S	A	B	C	D	G	E	F
---	---	---	---	---	---	---	---

Order of Visit:

S A B C D



Breadth-first-search (BFS)



Queue:

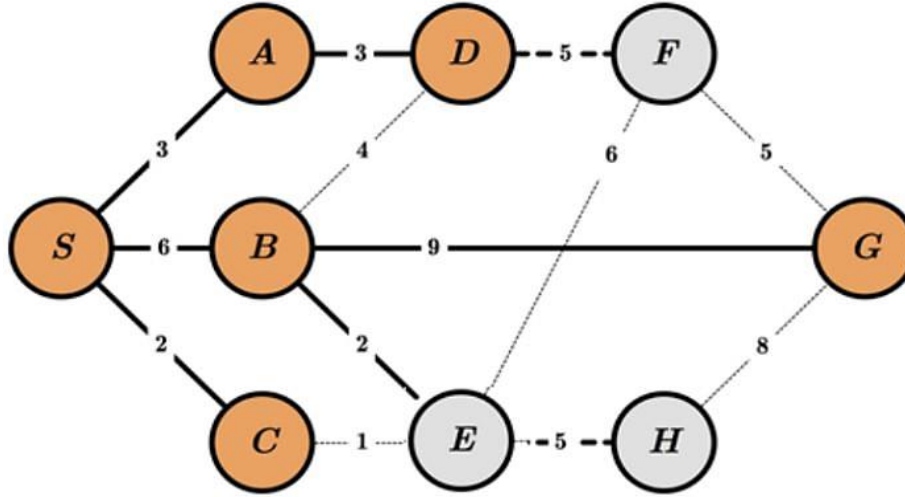
<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>G</i>	<i>E</i>	<i>F</i>
----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *B* *C* *D* *G*



Breadth-first-search (BFS)



Queue:

<i>S</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>G</i>	<i>E</i>	<i>F</i>
----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *B* *C* *D* *G*



Breadth-first-search (BFS)

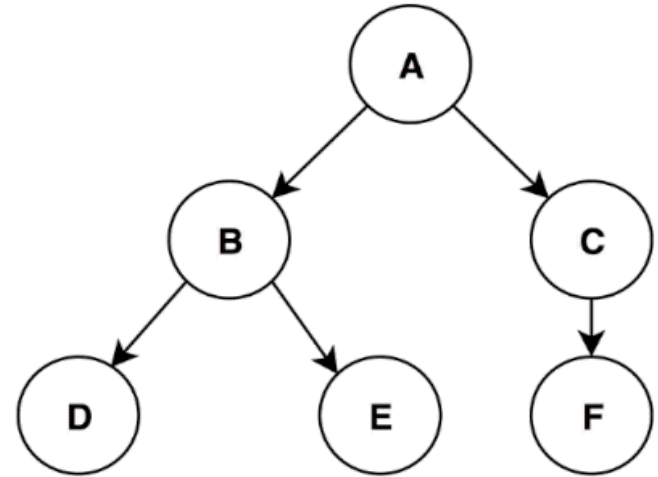
- BFS appears to have $O(|V| \times |E|)$ complexity due to nested loops but actually achieves **$O(|V| + |E|)$** through careful tracking.
- The outer while loop runs at most $|V|$ times since each vertex enters the queue exactly once, thanks to the visited set preventing repeats.
- The inner for loop examines edges, but crucially, each edge is examined only once from its source vertex or twice in undirected graphs.
- The total edge examinations sum to $O(|E|)$ across all vertices, not multiplied per vertex. **Thus, total work equals vertex operations ($|V|$) plus edge operations ($|E|$), making BFS efficient for large graphs.**

```
procedure BFS(G, s):  
    initialize empty queue Q  
    initialize visited set S  $\leftarrow \emptyset$   
  
    S.insert(s)           // Mark source visited  
    Q.enqueue(s)          // Enqueue for expansion  
  
    while Q  $\neq \emptyset$ :  
        v  $\leftarrow$  Q.dequeue()  
  
        for each neighbor u  $\in$  Adj[v]:  
            if u  $\notin$  S:           // State space pruning  
                S.insert(u)       // Prevent future rediscovery  
                Q.enqueue(u)      // Schedule for expansion
```

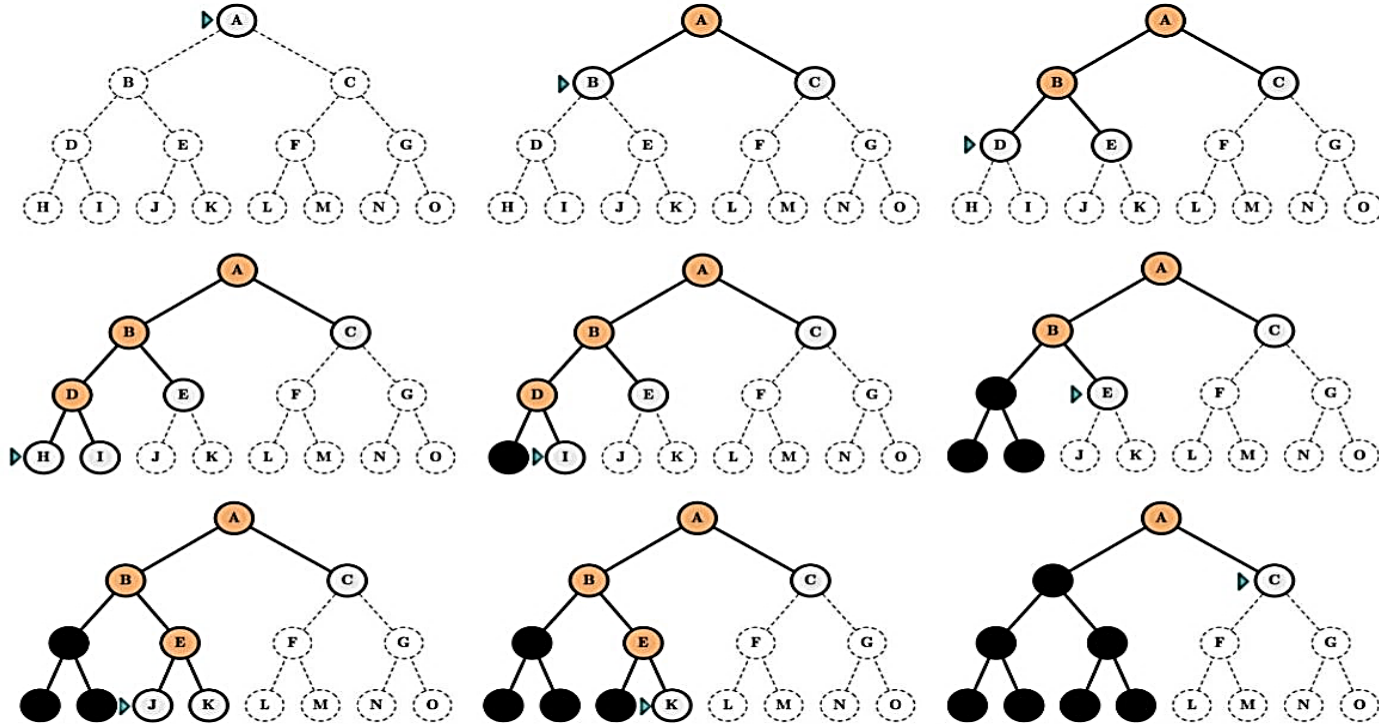


Depth First Search (DFS)

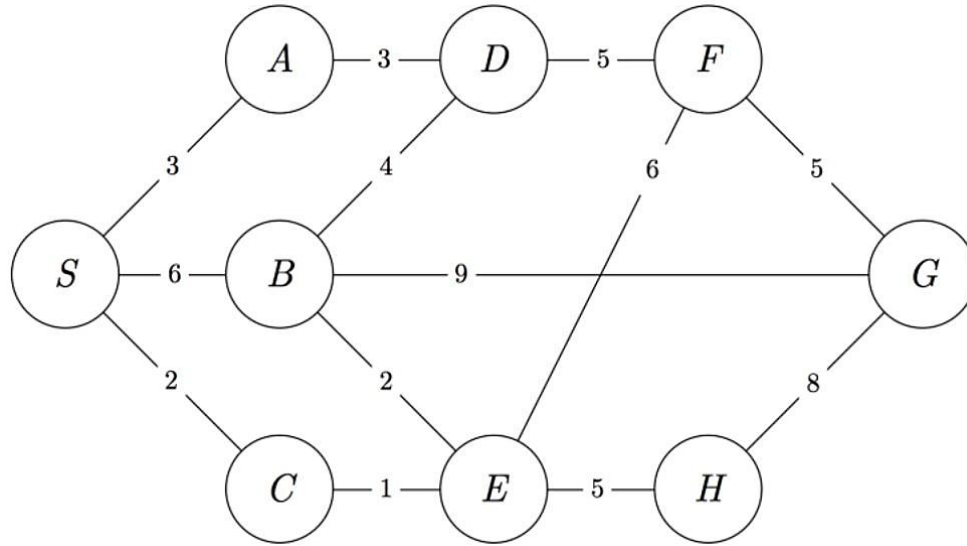
- After visit node v , DFS strategy proceeds along a path from v as deeply into the graph as possible before backing up.
- Avoiding Repeated Nodes: Keep track of all generated nodes. If the new node already exists, then discard the node.
- Uses a stack (Last input first output (LIFO)).



Depth First Search (DFS)



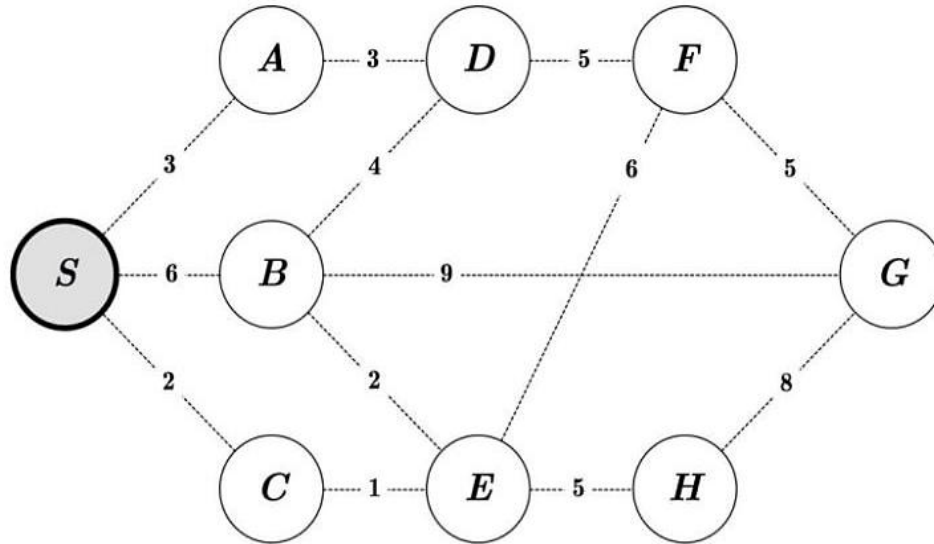
Depth First Search (DFS)



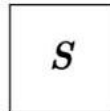
Question: What is the **order of visits of the nodes** and the **path returned** by DFS? Start node S and goal node G.



Depth First Search (DFS)



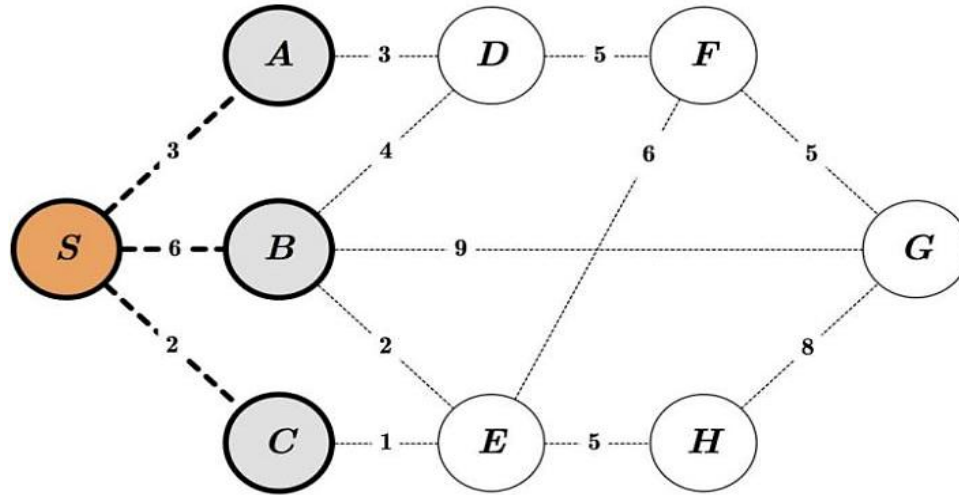
Stack:



Order of Visit:



Depth First Search (DFS)



Stack:

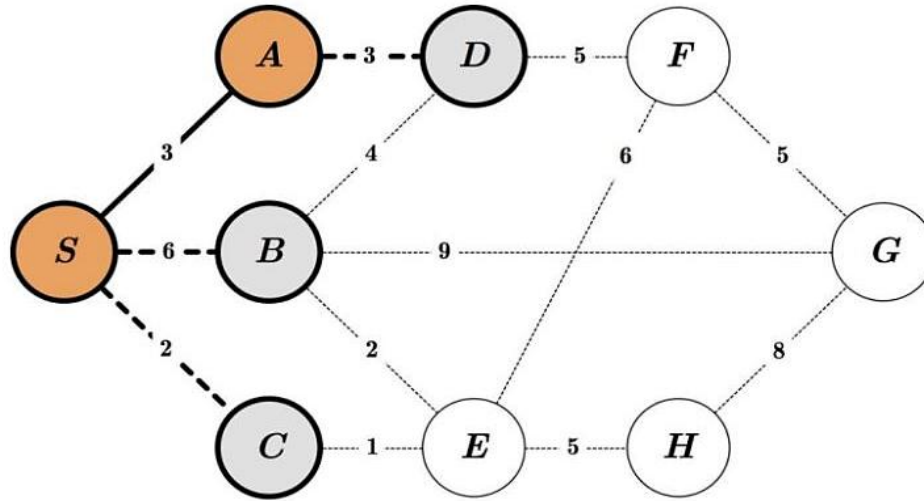
<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>
----------	----------	----------	----------

Order of Visit:

S



Depth First Search (DFS)



Stack:

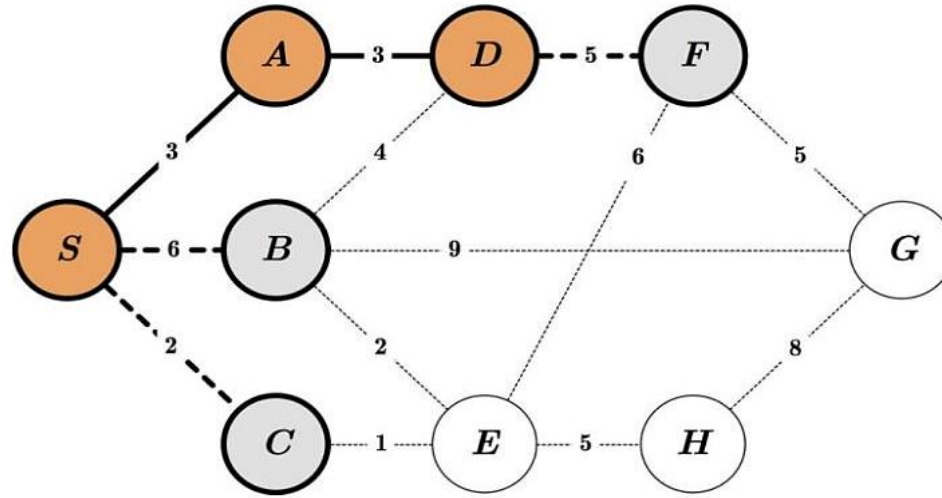
<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>
----------	----------	----------	----------	----------

Order of Visit:

S *A*



Depth First Search (DFS)



Stack:

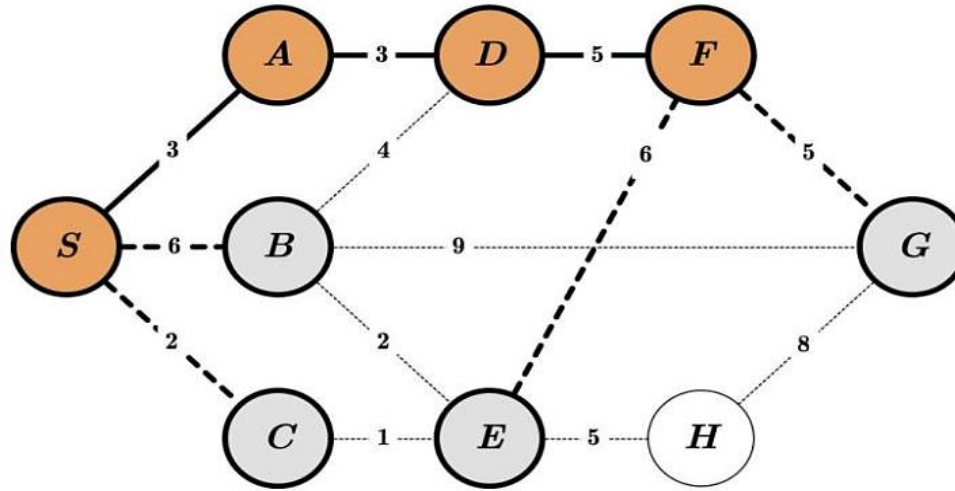
<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>
----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D*



Depth First Search (DFS)



Stack:

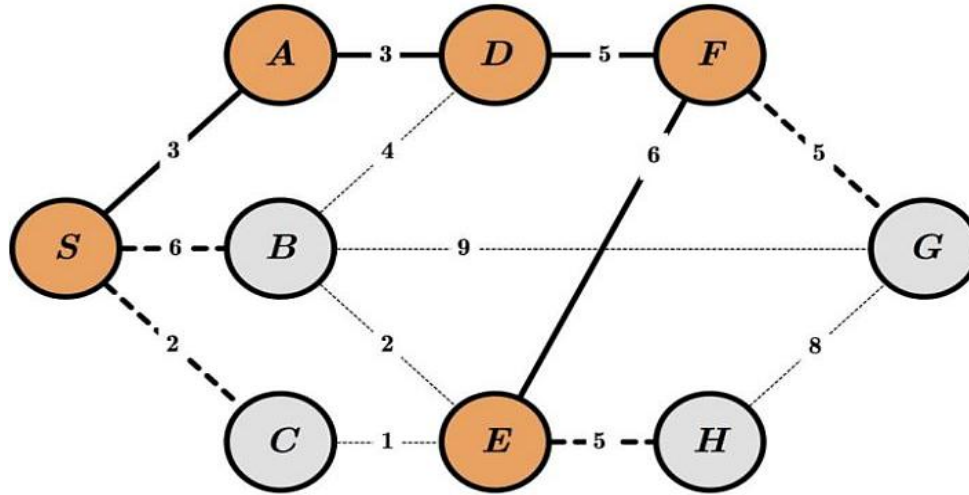
<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>
----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F*



Depth First Search (DFS)



Stack:

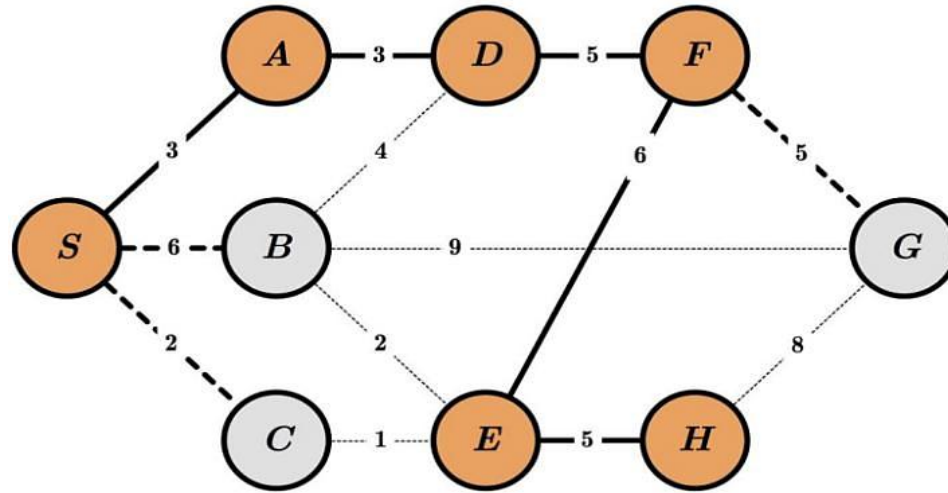
<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F* *E*



Depth First Search (DFS)



Stack:

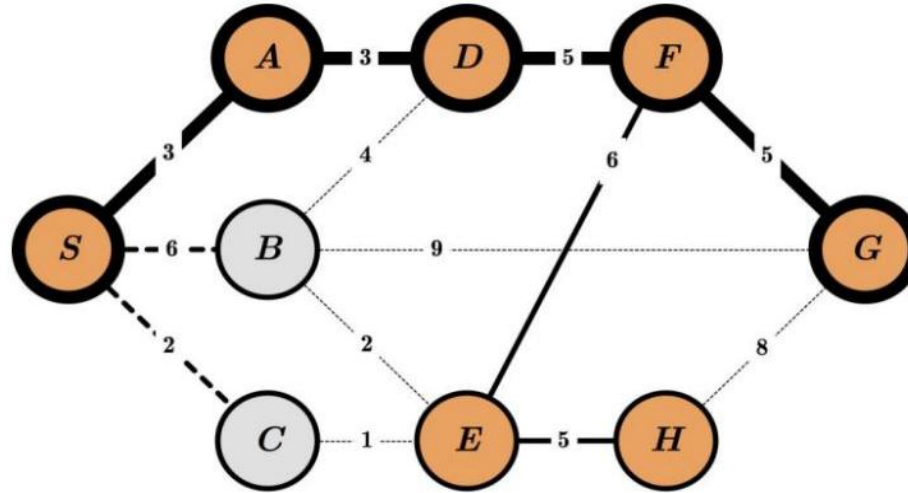
<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F* *E* *H*



Depth First Search (DFS)



Stack:

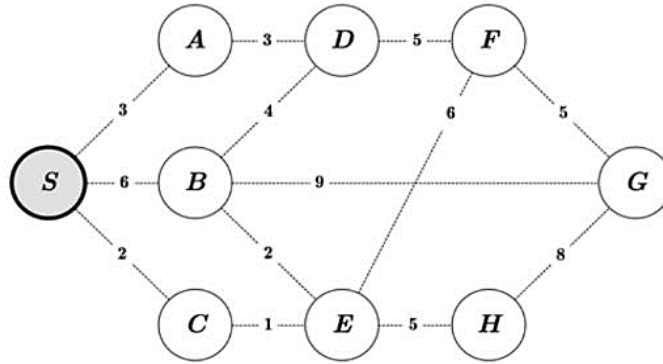
<i>S</i>	<i>C</i>	<i>B</i>	<i>A</i>	<i>D</i>	<i>F</i>	<i>G</i>	<i>E</i>	<i>H</i>
----------	----------	----------	----------	----------	----------	----------	----------	----------

Order of Visit:

S *A* *D* *F* *E* *H* *G*



Depth First Search (DFS)



Stack	S	C	B	A	D	F	E	G
Order of visit	S	A	D	F	G			

Another Solution:

Stack	S	A	B	C	E	F	H	G
Order of visit	S	C	E	H	G			



Depth First Search (DFS)

- DFS time complexity is $O(|V| + |E|)$, identical to BFS. Each vertex is visited exactly once, contributing $O(|V|)$ operations to the total time.
- Each edge is examined once when its source vertex is processed, adding $O(|E|)$ operations.
- The visited set prevents re-processing vertices, ensuring linear growth relative to graph size.

```
DFS_Simple(start):  
    stack = [start]  
    visited = set()  
  
    while stack:  
        current = stack.pop()  
  
        if current not visited:  
            mark current as visited  
  
            for each neighbor of current:  
                if neighbor not visited:  
                    stack.push(neighbor)
```



Any Question?

